Implementation of a Toolkit for Algebraic Module Encodings over $\mathbb{R}^n$ and Other Posets

TAMER-OP

Erik Mendes Novak

April 7th 2026

Posets and persistence modules

A *poset* is a set Q with a relation $\leq$ that is reflexive, antisymmetric, and transitive.

The main ambient posets in this talk are:

$$\mathbb{Z}^n, \qquad \mathbb{R}^n,$$

equipped with the coordinatewise order

$$a \leq b \iff a_i \leq b_i \text{ for all } i.$$

A *persistence module over* Q is a functor

$$M : Q \to \mathbf{Vect}_{\Bbbk}.$$

Equivalently:

- each $q \in Q$ carries a vector space $M(q)$,
- each relation $q \leq q'$ carries a linear map

$$M(q \leq q') : M(q) \to M(q'),$$

- and these maps compose functorially.

Upsets, downsets, and indicator modules

Let Q be a poset.

Order-theoretic regions

An *upset* $U \subseteq Q$ satisfies

$$q \in U,\ q \leq q' \Longrightarrow q' \in U.$$

A *downset* $D \subseteq Q$ satisfies

$$q' \in D,\ q \leq q' \Longrightarrow q \in D.$$

Indicator modules

To an upset U we associate the module $\Bbbk\{U\}$:

$$\Bbbk\{U\}(q) = \begin{cases} \Bbbk, & q \in U, \\ 0, & q \notin U, \end{cases}$$

with the obvious structure maps. Dually, a downset D gives $\Bbbk\{D\}$.

Upsts, downsets, and indicator modules

These indicator modules are the building blocks of presentations:

- upset indicators act like generators,
- downset indicators act like relations or cogenerators.

They are the persistence-module analog of the standard pieces used to present modules in commutative algebra.

Fringe presentations

A *fringe presentation* is a map

$$\varphi : \bigoplus_{i=1}^{m} \Bbbk\{U_i\} \longrightarrow \bigoplus_{j=1}^{r} \Bbbk\{D_j\},$$

where the U_i are upsets and the D_j are downsets.

The module of interest is the image

$$M = \mathrm{im}(\varphi).$$

After choosing bases on the indicator summands, φ is represented by a matrix

$$\Phi \in \Bbbk^{r \times m}.$$

At a parameter value q, only some upsets and some downsets are active:

$$I(q) = \{i : q \in U_i\}, \qquad J(q) = \{j : q \in D_j\}.$$

The fiber $M(q)$ is then computed from the submatrix

$$\Phi_{J(q),\, I(q)}.$$

Fringe presentations

So a fringe presentation packages a module by saying:

> *which generators are present where, which relations are present where, and how they are linearly coupled.*

Three presentation types used before encoding

Type	Ambient poset	Basic regions	Why it matters
Flange	$\mathbb{Z}^n$	flats and injectives	discrete input, adapted to finitely determined modules over $\mathbb{Z}^n$
PL box	$\mathbb{R}^n$	axis-aligned upsets/downsets cut by coordinate thresholds	fastest real-parameter backend; threshold grid gives efficient traversal
PL fringe	$\mathbb{R}^n$	general piecewise-linear upsets/downsets given by polyhedra	more general geometry; needed when boundaries are not axis-aligned

All three are mathematically the same kind of object:

- a presentation by indicator modules,
- but adapted to different ambient parameter spaces and geometries.

The encoding step converts all of them into the same output type: a finite poset P, an encoded module on P, and a classifier map

$$\pi : \text{ambient space} \to P.$$

Raw data as a fourth input type

Not every user begins with a presentation. Often the input is raw data:

point cloud, graph, image, graded complex, simplex tree.

A filtration specification turns that raw object into a finite graded object:

- simplicial or cubical complexes,
- graded chain or cochain complexes,
- and then modules or cohomology modules on a finite grid.

So there are really **four** front ends:

1. flange,
2. PL box presentation,
3. general PL fringe presentation,
4. raw filtered data.

The key design choice of the software is that these four very different inputs all flow into the same encoding-centered internal language.

Tameness and finite encoding

Finite encoding

A module M over a large poset Q is *finitely encoded* if there exist

- a finite poset P,
- a monotone classifier map $\pi : Q \to P$,
- and a P-module H,

such that

$$M \cong \pi^* H.$$

Tame module

A module is *tame* if it admits such a finite encoding.

Why is tameness good?

- It replaces an infinite or geometrically complicated parameter space by a finite one.
- It does *not* throw away the module: the original module is recovered by pullback.
- It makes exact algebra possible: kernels, cokernels, Hom, Ext, Tor, mapping cones, and more can all be computed on the finite encoded object.

So the encoder is not producing a summary invariant. It is producing a finite, computable model of the module itself.

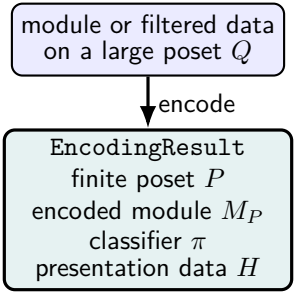
The computational question

We want to replace a module on a large indexing poset by finite data

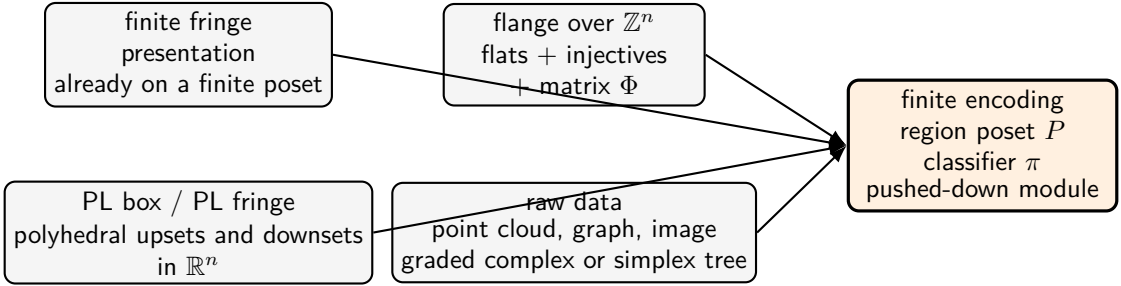
$$M \cong \pi^* H, \qquad \pi : Q \to P,$$

where P is finite and H is a module on that finite poset.

- the front end may live on $\mathbb{Z}^n$, $\mathbb{R}^n$, a finite poset, or come from filtered data;
- the output must be reusable for algebra, invariants, serialization, and visualization;
- this is why the workflow returns a full encoded object, not just a summary.



Four front end input types



The implementation keeps the front end heterogeneous, then forces all four inputs into one finite internal language.

The universal idea: signatures and constant regions

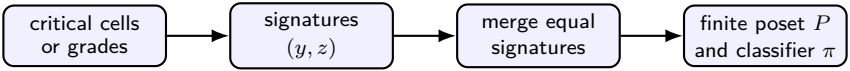
For a presentation

$$\varphi : \bigoplus_i \Bbbk\{U_i\} \longrightarrow \bigoplus_j \Bbbk\{D_j\},$$

we classify each parameter value q by the signature

$$\sigma(q) = (y(q), z(q)), \qquad y_i(q) = 1 \iff q \in U_i, \qquad z_j(q) = 1 \iff q \notin D_j.$$

Equal signatures mean the same active generators and relations, hence the same local presentation matrix.



This signature-first pattern is the common core of every backend.

Encoding a flange over $\mathbb{Z}^n$

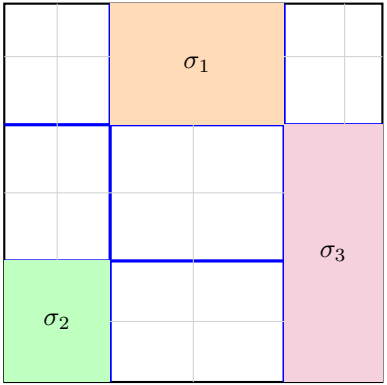
A flange input consists of flats, injectives, and a matrix Φ. The encoder uses the fact that membership can change only across finitely many coordinate thresholds.

- extract critical coordinates on each axis;
- cut $\mathbb{Z}^n$ into slab cells;
- evaluate signatures on slab representatives;
- merge equal signatures into a finite signature poset;
- push the presentation down to a finite fringe module on P.

Implementation details that matter in practice: bit-packed signatures, reusable slab traversal, and common encoding for several flanges at once.

Encoding a flange over $\mathbb{Z}^n$

$\mathbb{Z}^2$ (window)



threshold cuts from
flats and injectives

$$\sigma(g) = (y(g), z(g))$$

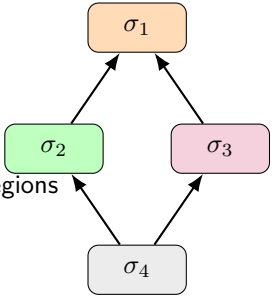
$$y_i(g) = 1 \iff g \in F_i$$

$$z_j(g) = 1 \iff g \notin E_j$$

$$\pi(g) = \sigma(g)$$

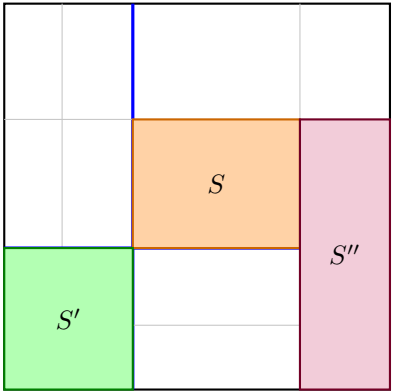
collapse equal-signature regions

finite encoding poset P



Encoding flanges over $\mathbb{Z}^n$: slab cells

$\mathbb{Z}^2$ (finite window)



threshold cuts induced by flats and injectives

Signature at a lattice point g

$$\sigma(g) = (y(g), z(g))$$

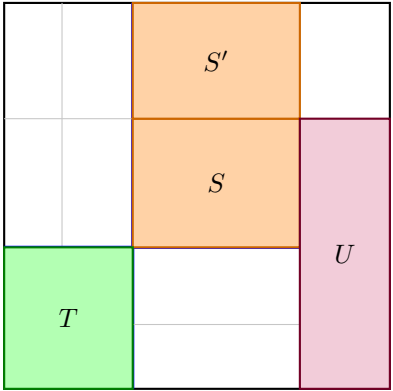
$$y_i(g) = 1 \iff g \in F_i, z_j(g) = 1 \iff g \notin E_j$$

Why slab cells matter

Thresholds are exactly the coordinates where membership in some flat or injective can change. Inside one slab cell, all these membership bits are constant. So each slab cell has a well defined signature.

Encoding flanges over $\mathbb{Z}^n$: quotient by signature

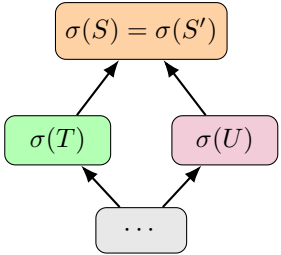
$\mathbb{Z}^2$ (finite window)



$\pi(g) = \sigma(g)$

If $\sigma(S) = \sigma(S')$, then S and S' represent the same encoded region.
The encoding is therefore not the slab grid itself, but its quotient by realized signatures.

finite encoding poset P



one node for each realized signature class

slab cells with equal signatures are merged

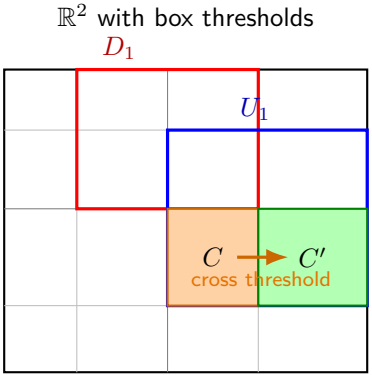
Encoding PL box presentations

For axis-aligned boxes in $\mathbb{R}^n$, the geometry is simpler and the traversal can be specialized.

- collect all lower and upper coordinate thresholds;
- build the rectangular cell grid they induce;
- traverse cells in odometer order;
- update signatures incrementally when a threshold is crossed;
- build a direct cell-to-region lookup table.

This backend is faster because it avoids generic polyhedral feasibility and uses mixed-radix indexing for point location.

Encoding PL box presentations: threshold cells



rectangular cells induced by coordinate thresholds

Cell signature

$\sigma(x) = (y(x), z(x))$
$y_i(x) = 1 \iff x \in U_i$
$z_j(x) = 1 \iff x \notin D_j$

Key point

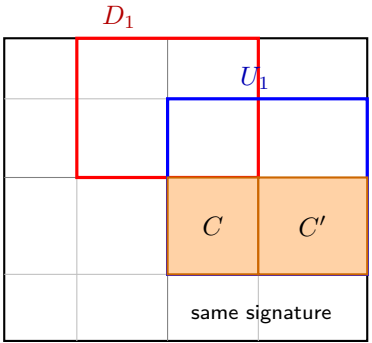
Each cell lies entirely inside or outside every box generator. Therefore the signature is constant on each cell.

Traversal idea

Visit cells in odometer order. Crossing one threshold changes only a few membership bits, so signatures are updated sparsely instead of recomputed from scratch.

Encoding PL box presentations: quotient by signature

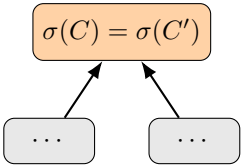
$\mathbb{R}^2$ with box thresholds



$\pi(x) = \sigma(x)$

If $\sigma(C) = \sigma(C')$ then C and C' belong to the same encoded region.

finite encoding poset P



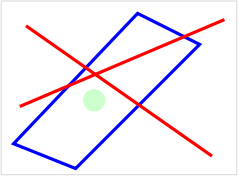
one node for each realized signature class

cells with equal signatures are merged

Encoding general PL fringes

For genuine polyhedral upsets and downsets, the backend is signature-first rather than arrangement-first.

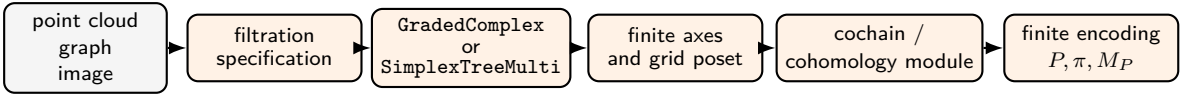
- store each convex piece by exact halfspaces over $\mathbb{Q}$;
- avoid building the full arrangement;
- "inside a union" is handled by choosing a piece;
- "outside a union" is handled by choosing a facet inequality to violate;
- solve exact feasibility only for candidate signatures;
- cache witness points and float surrogates for later point location.



realize only the
signatures that are feasible

From raw data to an encoded module

Raw data passes through a finite graded object before encoding.



The key point is that data-derived modules and presentation-derived modules meet at the same endpoint: a finite encoded module on a finite poset.

What the classifier stores, and why it matters

The classifier is not just a formal map $\pi : Q \to P$. It is a compiled object used repeatedly by later routines.

Stored information	Why it matters
region representatives, axes, cell lookup	fast point location and batched queries
realized signatures and finite poset structure	exact pushed-down module and naturality of maps
region geometry: weights, bounding boxes, adjacency	weighted invariants, support summaries, visualization
cache fingerprints and compiled metadata	reuse across repeated computations and common encodings

This is why the output is an `EncodingResult` rather than a bare finite poset.

Takeaways

1. Four very different front ends are accepted: finite fringes, flanges, PL presentations, and raw filtered data.
2. All of them are turned into the same internal object: a finite poset, a finite module on that poset, and a compiled classifier map.
3. The universal mechanism is signature-based: identify constant regions, realize only the signatures that occur, then push the module down.
4. The backend changes with the geometry:
   - flange: slab cells in $\mathbb{Z}^n$;
   - PL box: rectangular traversal with sparse updates;
   - general PL: exact feasibility on candidate signatures;
   - raw data: graded complexes or simplex trees, then cohomology on a finite grid.
5. Once the encoding exists, all later algebra and invariants work from the same finite object.

Selected references

- E. Miller, *Homological algebra of modules over posets*, 2020.
- M. Lesnick, *The theory of the interleaving distance on multidimensional persistence modules*, 2015.
- M. Lesnick and M. Wright, *Interactive visualization of 2-D persistence modules*, 2015.
- M. Kerber, M. Lesnick, and S. Oudot, *Exact computation of the matching distance on 2-parameter persistence modules*, 2020.
- M. Carriere and A. Blumberg, *Multiparameter persistence image for topological machine learning*, 2020.